

MetrCheck AI

Complete System Architecture, Code Flow & Technical Reference Manual

EXECUTIVE SUMMARY:

MetrCheck AI is an enterprise-grade, automated statutory compliance auditing and decision-support platform engineered specifically for pre-packaged commodities sold across retail and e-commerce platforms in India. Operating under the strict statutory mandates of the **Legal Metrology (Packaged Commodities) Rules, 2011** and the **Food Safety and Standards (Packaging and Labelling) Regulations, 2020**, the system bridges the gap between raw packaging artwork/images and verifiable legal compliance through deterministic multi-stage AI reasoning.

The system ingests multi-panel product photographs (Front, Back, Sides) or e-commerce listing descriptions, executes high-resolution computer vision preprocessing (EXIF, perspective rectification, Lanczos upscaling), performs multi-pass OCR (PaddleOCR PP-OCRv4 + Tesseract fallback), extracts statutory packaging declarations via contextual pattern matching with automatic character repair, validates them against 14 codified statutory rules, computes explainable 0–100 compliance scores, produces prioritized manufacturer corrective guidance, estimates statutory legal penalties under Legal Metrology Act Sections 36 & 38, generates court-ready show-cause notices, and exports multi-format inspection dossiers (ReportLab PDF, openpyxl XLSX, CSV, JSON).

1. System Architecture & Component Topology

MetrCheck AI is engineered with a strict 5-layer decoupled architecture ensuring complete separation of concerns, high throughput asynchronous processing, zero external runtime internet dependencies, and verifiable test isolation.

METRCHECK AI — SYSTEM ARCHITECTURE & COMPONENT TOPOLOGY

Modular Layer Separation: UI, API Gateway, Computer Vision, Rule Engine, and Storage

LAYER 1: PRESENTATION LAYER (React 19 + TypeScript + Vite + Tailwind CSS v4)

Pages & Dashboards

Dashboard.tsx, Analyze.tsx (Multi-panel),
AnalyzeListing.tsx, Results.tsx,
History.tsx, DemoCases.tsx, AdminUsers.tsx

Visual Components

EvidenceViewer.tsx (2D BBoxes),
ScoreCircle, StatusBadge, SeverityBadge,
LoadingSkeleton, Card, ThemeToggle

State & Context

AuthContext (JWT & Bearer Tokens),
RoleContext (Admin/Officer/Merchant),
ThemeContext (Dark/Light mode)

LAYER 2: REST API GATEWAY & AUTHENTICATION (FastAPI Async Framework)

API Routers (/api/*)

analyze.py, ocr.py, extract.py,
compliance_routes.py, history.py,
demo.py, health.py, report.py, enforcement.py

Auth & Security (auth/)

PBKDF2-HMAC-SHA256 (200k iter),
Signed HMAC-SHA256 Bearer tokens,
Role-based guards (require_roles)

Middleware & Config

CORS Middleware, StaticFiles (/uploads),
Lifespan DB setup, Pydantic-Settings
Config with Test Isolation guards

LAYER 3: COMPUTER VISION, OCR & EXTRACTION ENGINE

OCR Subsystem (ocr/)

PaddleOCR PP-OCRv4 (DBNet+SVTR),
Tesseract Engine fallback,
Multi-pass Preprocessing & Orientation

Region & Quality (ocr/)

RegionDetector (Header/Mid/Stamp/Table),
QualityAssessor (Blur, Brightness, Contrast),
Text Cleaner & FSSAI Unit Repair

Extraction (extraction/)

LocalExtractor (High-recall regex),
Contextual validator, Confidence scoring,
FSSAI 14-digit state/type decoder

LAYER 4: STATUTORY COMPLIANCE & LEGAL ENFORCEMENT ENGINE

Rule Registry (compliance/)

LM-001 to LM-009 (Packaged Commodities),
FS-001 to FS-005 (FSSAI Regulations),
Rule 12 Font Height Calculator

Scorer & Recommendations

Weighted Compliance Formula,
Verdict Categorization (Pass/Warn/Fail),
Deterministic Corrective Action Engine

Enforcement (enforcement/)

LM Act 2009 Sec 36/38 Penalty Estimator,
Statutory Show-Cause Notice Builder,
Advisory sanction guidelines

LAYER 5: DATA PERSISTENCE & MULTI-FORMAT EXPORT ENGINE

SQLite Storage (database/)

aiosqlite async database (metrc_check.db),
'analyses' table (JSON-serialized payloads),
'users' table (salted hashes & RBAC)

Audit Dossiers (services/)

ReportLab PDF Generator (multi-page),
openpyxl XLSX (3-sheet formatted Excel),
Standard CSV & JSON ingestion endpoints

Orchestration (services/)

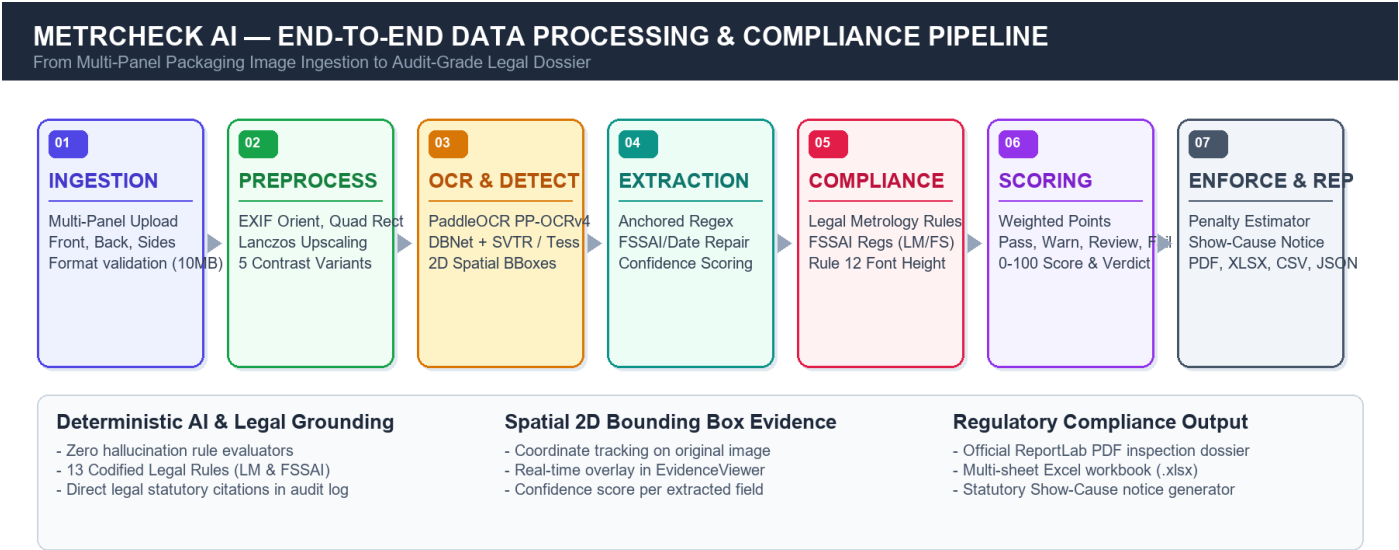
analysis_service.py (pipeline coordination),
image_service.py (disk persistence),
runner.py (multi-port supervisor)

Layer	Key Modules & Technologies	Primary Architectural Responsibility
Layer 1: Presentation	React 19, TypeScript, Vite, Tailwind CSS v4, Lucide React, Recharts	Multi-panel drag-and-drop upload, 2D EvidenceViewer with bounding box overlays, compliance gauges, listing text paste screening, admin console.

Layer 2: REST Gateway	FastAPI, Pydantic v2, Uvicorn, PBKDF2-HMAC-SHA256 Auth, HMAC-SHA256 Bearer	Asynchronous REST API routing, Bearer token authentication, Role-Based Access Control (RBAC), CORS middleware, input payload validation.
Layer 3: Vision & OCR	PaddleOCR (PP-OCRv4 DBNet+SVTR), pytesseract, OpenCV, Pillow, NumPy	EXIF orientation, quad perspective rectification, dynamic Lanczos scaling, 5 binary filter passes, 2D spatial coordinate tracking, OCR text cleaner & repair.
Layer 4: Compliance & Legal	RuleRegistry, LM-001..LM-009, FS-001..FS-005, Scorer, Penalty Estimator	Deterministic statutory evaluation, Rule 12 font height calculation, weighted point scoring, prioritized corrective actions, Sec 36/38 penalty brackets.
Layer 5: Storage & Export	aiosqlite (SQLite), ReportLab PDF, openpyxl XLSX, CSV, JSON	Persistent screening audit logs, role-gated user database, multi-sheet formatted Excel generation, official multi-page PDF inspection reports.

2. End-to-End Data Processing Pipeline & Code Execution Flow

Every packaging analysis undergoes a deterministic, high-throughput 7-stage execution pipeline coordinated by `backend/services/analysis_service.py`.



06

SCORING

Weighted Points
Pass, Warn, Review, Fail
0-100 Score & Verdict

07

ENFORCE & REP

Penalty Estimator
Show-Cause Notice
PDF, XLSX, CSV, JSON

Deterministic AI & Legal Grounding

- Zero hallucination rule evaluators
- 13 Codified Legal Rules (LM & FSSAI)
- Direct legal statutory citations in audit log

Spatial 2D Bounding Box Evidence

- Coordinate tracking on original image
- Real-time overlay in EvidenceViewer
- Confidence score per extracted field

Regulatory Compliance Output

- Official ReportLab PDF inspection dossier
- Multi-sheet Excel workbook (.xlsx)
- Statutory Show-Cause notice generator

Stage 1: Multi-Panel Ingestion & Image Validation

API endpoint `POST /api/analyze` receives 1 to 4 upload files labeled as 'Front', 'Back', 'Side 1', 'Side 2'. `utils/validators.py:validate_image_file()` enforces MIME type whitelist (PNG, JPEG, JPG, WebP) and 10MB maximum file size. `services/image_service.py:process_and_save_image()` writes sanitized files to `backend/uploads/` with unique UUIDs.

Stage 2: Advanced Computer Vision Preprocessing

`ocr/preprocessing.py:ImagePreprocessor` executes non-destructive image enhancements: (1) *EXIF Transposition* to normalize rotated phone snapshots; (2) *Perspective Rectification* using OpenCV Canny edges, contour polygon approximation, and 4-point homography warp; (3) *Dynamic Lanczos Upscaling* (scale factor up to 2.8x for labels under 1200px); (4) Generation of 5 targeted multi-pass variants (Grayscale, Otsu Adaptive, CLAHE High-Contrast, Morphological Opening, and Deskewed).

Stage 3: Multi-Pass OCR & Spatial Bounding Box Extraction

`ocr/factory.py:get_ocr_engine()` initializes PaddleOCR (PP-OCRv4 deep learning with DBNet detection & SVTR recognition) or Tesseract OCR. Extracts recognized text lines, confidence scores, and 2D bounding boxes [ymin, xmin, ymax, xmax] for every word. `ocr/quality.py` computes blur (Laplacian variance), brightness, and contrast diagnostics to warn users of degraded photography.

Stage 4: OCR Normalization & Character Repair

`ocr/cleaner.py:clean_ocr_text()` eliminates unicode replacement artifacts while strictly preserving currency symbols (₹, Rs.), decimals, units, and dates. `ocr/repair.py` applies contextual heuristics: auto-corrects FSSAI transposed letters (O/I/I/S/B/G/Z \$ o\$ 0/1/1/5/8/6/2), fixes Net Qty '9' \$ o\$ 'g', and cleans MRP/date delimiters.

Stage 5: Structured Information Extraction

`extraction/extractor.py:LocalExtractor` runs anchored statutory regex patterns (`patterns.py`) to extract: Product Name, Brand, Manufacturer/Packer with PIN code, Net Quantity, MRP, Mfg/Exp/Use-By Dates, Batch Number, FSSAI Licence, Consumer Care Phone/Email, Country of Origin, Ingredients, Nutrition table, and Barcodes. Computes dynamic confidence scores per declaration and decodes FSSAI state codes.

Stage 6: Codified Statutory Compliance Evaluation

`compliance/engine.py:ComplianceEngine` checks all 14 statutory rules (LM-001..009 and FS-001..005). `compliance/rules/legal_metrology.py:compute_font_size_and_readability()` measures pixel-to-millimeter font height against Rule 12 statutory minimums based on package net quantity. `compliance/scorer.py:calculate_score()` evaluates weighted points (PASS=1.0, WARN=0.5, REVIEW=0.85, FAIL=0.0) to generate the final 0–100 score and categorical status.

Stage 7: Report & Enforcement Artefact Generation

`compliance/recommendations.py` builds prioritized manufacturer corrective actions with statutory citations. `enforcement/penalties.py` calculates Legal Metrology Act Sec 36/38 penalty brackets and generates court-ready show-cause notices. `services/report_service.py` and `backend/api/report.py` generate downloadable PDF, XLSX, CSV, and JSON audit dossiers. Record is saved to SQLite analyses table.

3. Statutory Compliance Rules Matrix & Scoring Mathematics

MetrCheck AI codifies 14 statutory compliance rules split across two regulatory domains: **Legal Metrology** (Department of Consumer Affairs) and **FSSAI** (Food Safety and Standards Authority of India).

METRCHECK AI — STATUTORY COMPLIANCE & SCORING DECISION MATRIX

Codified Rules, Weighted Point Calculation, and Multi-Level Threshold Classification

1. LEGAL METROLOGY (9 Rules)

LM-001: Mfr/Packer/Importer
Rule 6(1)(a) — Physical address & PIN code

LM-002: Common / Generic Name
Rule 6(1)(b) — Identity of commodity

LM-003: Net Quantity & Units
Rule 6(1)(c) — Standard metric units (g, kg, ml)

LM-004: Maximum Retail Price
Rule 6(1)(e) — MRP inclusive of all taxes

LM-005: Consumer Care Info
Rule 6(1)(f) — Helpline phone, email & address

LM-006: Country of Origin
Rule 6(1)(d) — Mandatory for all goods

LM-007: Unit Sale Price (USP)
Rule 6(11) — Price per g/kg/ml for consumers

LM-008: Date of Mfg / Pre-pack
Rule 6(1)(d) — Month & Year marking

LM-009: Misleading & Rule 12
Rule 6(1)(e) & Rule 12 min font height

2. FSSAI FOOD REGS (5 Rules)

FS-001: 14-Digit FSSAI Lic/Logo
FSS Regulations — 14 digits & state decoding

FS-002: Standardized Food Name
FSS Act — Specific commodity denomination

FS-003: List of Ingredients
FSS Labelling — Descending order of weight

FS-004: Nutrition Facts Panel
FSS Labelling — Energy, Carbs, Sugar, Fats

FS-005: Date Marking / Expiry
FSS Labelling — Best Before / Use-By date

Applicability Condition
Automatically activated for food commodities

Non-Food Handling
Marked NOT_APPLICABLE (excluded from score)

OCR Character Repair
Auto-recovers 'FSSAI' and swapped digits

State Code Parsing
Decodes Central vs State Licensing jurisdiction

3. SCORING & VERDICTS

Weighted Scoring Formula
Earned Points / Applicable Rules * 100

PASS Weight = 1.0 (100%)
Full compliance verified with high confidence

WARNING Weight = 0.5 (50%)
Minor defect / non-standard abbreviation

NEEDS REVIEW Weight = 0.85
Uncertain OCR / requires visual inspection

FAIL Weight = 0.0 (0%)
Missing mandatory statutory declaration

NOT_APPLICABLE = Excluded
Excluded completely from total denominator

Status: COMPLIANT
Score >= 80% & Zero statutory failures

Status: REVIEW REQUIRED
Score < 80% or inspection flags present

Status: POTENTIAL NON-COMPL
Triggered if ANY statutory FAIL occurs

STATUTORY COMPLIANCE SCORING FORMULA:

$$Score = \left(\frac{1.0 \times N_{PASS} + 0.5 \times N_{WARNING} + 0.85 \times N_{NEEDS_REVIEW} + 0.0 \times N_{FAIL}}{N_{Applicable\ Rules}} \right) \times 100$$

Classification Verdict Logic:

- POTENTIAL NON-COMPLIANCE:** Assigned if $N_{FAIL} \geq 1$ (any confirmed mandatory declaration missing/illegal).
- REVIEW REQUIRED:** Assigned if $N_{FAIL} = 0$ but $N_{NEEDS_REVIEW} \geq 1$ or overall score < 80.0%.
- COMPLIANT:** Assigned if $N_{FAIL} = 0$, $N_{NEEDS_REVIEW} = 0$, and overall score $\geq 80.0\%$.
- Note:** Rules evaluated as **NOT_APPLICABLE** (e.g. FSSAI rules on non-food goods) are completely excluded from the denominator.

Rule ID	Domain	Statutory Source & Clause	Requirement & Verification Criteria	Severity
LM-001	Legal Metrology	Rule 6(1)(a), LM (PC) Rules 2011	Name & complete physical address of Manufacturer, Packer, or Importer with valid PIN code.	HIGH
LM-002	Legal Metrology	Rule 6(1)(b), LM (PC) Rules 2011	Generic / common commodity name conspicuously declared on principal display panel.	MEDIUM
LM-003	Legal Metrology	Rule 6(1)(c) & Rule 12, LM Rules	Net quantity in standard metric units (g, kg, ml, l, pcs, N). Flags non-standard units (gms, kgs, ltr).	HIGH
LM-004	Legal Metrology	Rule 6(1)(e), LM (PC) Rules 2011	Maximum Retail Price (MRP) in INR with explicit 'Inclusive of all taxes' declaration.	HIGH
LM-005	Legal Metrology	Rule 6(1)(f), LM (PC) Rules 2011	Consumer Care Helpline details: Contact name, physical address, toll-free telephone, and email.	HIGH
LM-006	Legal Metrology	Rule 6(1)(d), LM (PC) Rules 2011	Country of Origin clearly declared on packages (e.g. 'Made in India', 'Country of Origin: USA').	MEDIUM
LM-007	Legal Metrology	Rule 6(11), LM (PC) Rules 2011	Unit Sale Price (USP) per g/kg/ml for packages containing more than 1 unit/kg.	LOW
LM-008	Legal Metrology	Rule 6(1)(d), LM (PC) Rules 2011	Month and Year of Manufacture, Pre-packing, or Import in valid MM/YYYY or MM/YY format.	HIGH
LM-009	Legal Metrology	Rule 6(1)(e) & Rule 12 Font Height	Misleading statement detection and Net Quantity font height verification against statutory thresholds.	MEDIUM
FS-001	FSSAI	FSS (Packaging & Labelling) 2020	14-digit FSSAI licence number starting with 1 or 2 alongside the official FSSAI logo.	HIGH
FS-002	FSSAI	FSS (Packaging & Labelling) 2020	Standardized food product denomination indicating true nature and physical state of food.	MEDIUM

FS-003	FSSAI	FSS (Packaging & Labelling) 2020	Ingredients listed under heading 'Ingredients:' in descending order of in-going weight or volume.	MEDIUM
FS-004	FSSAI	FSS (Packaging & Labelling) 2020	Nutrition Information Panel per 100g/serving: Energy (kcal), Protein, Carbohydrate, Total Sugars, Fats.	HIGH
FS-005	FSSAI	FSS (Packaging & Labelling) 2020	Date marking: 'Best Before' or 'Expiry Date' with day/month/year or relative shelf life declaration.	HIGH

4. Security Architecture & Role-Based Access Control (RBAC)

MetrCheck AI implements an enterprise-grade security sub-system built entirely on Python standard library cryptography (zero third-party dependencies like PyJWT or passlib).

METRCHECK AI — SECURITY & ROLE-BASED ACCESS CONTROL (RBAC)
Cryptographic Auth Architecture, Role Hierarchy, and Access Permission Matrix

1. SYSTEM ADMINISTRATOR (ADMIN)

Full Administrative Control & User Governance

User Account Creation

Provision new Officers & Merchants (/api/auth/users)

User Modification & Deletion

Update profiles, reset passwords, delete accounts

Global History Management

Full audit log access & complete database purge

System Configuration

Configure OCR engines, storage paths & test isolation

Root Privileges

Protected: Admin account cannot be altered by lower roles

2. ENFORCEMENT OFFICER (DoCA)

Regulatory Inspection & Enforcement Workflows

Package & Listing Screening

Upload multi-panel images or paste listing text

Evidence & Bounding Boxes

Review 2D spatial overlays & OCR confidence scores

Penalty Calculator (Sec 36/38)

Estimate statutory fines (1st vs repeat offences)

Show-Cause Notice Generator

Export formatted statutory notices for violators

Audit Report Downloads

Export multi-page PDF, XLSX, CSV, JSON dossiers

3. MERCHANT / PUBLIC USER

Pre-Market Compliance Self-Screening

Self-Service Label Audit

Upload packaging artwork to detect omissions

E-Commerce Listing Check

Verify catalogue descriptions before publishing

Corrective Action Guidance

Receive step-by-step label rectification steps

Rule Registry Browser

Explore Legal Metrology & FSSAI statutory standards

Isolated Permissions

No access to enforcement tools or admin console

Cryptographic Primitives (backend/auth/security.py):

- **Password Storage:** Passwords are never stored in plaintext. They are salted with `os.urandom(16)` and hashed via **PBKDF2-HMAC-SHA256** with **200,000 iterations**. Verification uses `hmac.compare_digest` to prevent timing attacks.
- **Token Generation & Signing:** State-less bearer tokens are signed with **HMAC-SHA256** using the server's `SECRET_KEY`. Tokens encode username, role, issue time, and expiration timestamp (default 480 minutes / 8 hours).
- **Role Hierarchy & Route Guards:** `require_roles(*roles)` dependency injects role enforcement at the FastAPI router level.

Feature / Endpoint Capability	ADMIN	ENFORCEMENT_OFFICER	MERCHANT_PUBLIC
Multi-Panel Packaging Image Upload & OCR Analysis	Yes	Yes	Yes
E-Commerce Listing Text Screening (/api/extract, /api/compliance/check)	Yes	Yes	Yes
Rule Registry Inspection (/api/compliance/rules)	Yes	Yes	Yes
Download PDF / XLSX / CSV / JSON Compliance Reports	Yes	Yes	Yes
View Inspection Audit History & Trends (/api/history, /api/stats)	Yes	Yes	No
LM Act 2009 Sec 36/38 Statutory Penalty Estimator (/api/enforcement/penalty)	Yes	Yes	No
Court-Ready Show-Cause Notice Generator (/api/enforcement/notice)	Yes	Yes	No
Delete Individual Analysis Record (DELETE /api/history/{id})	Yes	Yes	No
Purge All Audit History Records (DELETE /api/history)	Yes	No	No
User Management Console (Create/Edit/Delete Accounts, Reset Passwords)	Yes	No	No

5. Complete Codebase Directory & File Inventory

Complete technical mapping of all source files in the repository across backend micro-services, OCR modules, compliance rules, database layers, and the React frontend.

Path / Module	Type	Functional Description & Exported Symbols
backend/main.py	Backend	FastAPI app entrypoint, lifespan startup (init_db), CORS middleware, static mounts (/uploads), router inclusions.
backend/config.py	Backend	Pydantic-Settings Settings class. Controls OCR_ENGINE, TESSERACT_PATH, DATABASE_PATH, UPLOAD_DIR, TEST_MODE, and test isolation checks.
backend/database/db.py	Database	aiosqlite async operations: init_db(), save_analysis(), get_analyses(), delete_analysis(), create_user(), update_user(), delete_user(), search_analyses(), seed_default_users().
backend/models/schemas.py	Schemas	Pydantic models: OCRWord, OCRResult, ProductInfo, ComplianceCheck, ComplianceResult, Recommendation, ProductImageEvidence, FontSizeAnalysis, AnalysisResponse, HistoryItem.
backend/ocr/factory.py	OCR	get_ocr_engine() singleton factory. Dynamically selects PaddleOCREngine, TesseractEngine, or auto fallback.
backend/ocr/paddle_engine.py	OCR	PaddleOCR PP-OCRv4 implementation with OneDNN Windows flags workaround, bounding box extraction, and confidence aggregation.
backend/ocr/tesseract_engine.py	OCR	pytesseract engine with multi-pass binarization and PSM configuration.
backend/ocr/preprocessing.py	CV Pipeline	EXIF transpose, perspective/quad detection and homography rectification, dynamic Lanczos upscaling (1.6x-2.8x), and 5 filter variants.
backend/ocr/cleaner.py	CV Pipeline	clean_ocr_text(): Strips stray punctuation while preserving currency (₹), numbers, dates, units, and FSSAI keywords.
backend/ocr/repair.py	CV Pipeline	Specialized repair heuristics: repair_fssai_license() (swapped glyphs), repair_net_quantity(), repair_mrp(), repair_date(), repair_batch_number().
backend/ocr/quality.py	CV Pipeline	assess_image_quality(): Computes blur score (Laplacian variance), brightness, contrast, and resolution diagnostics.
backend/ocr/regions.py	CV Pipeline	RegionDetector: Contour & morph-based segmentation for Brand Header, Middle Statutory, Bottom Stamp, and Nutrition tables.
backend/extraction/extractor.py	Extraction	LocalExtractor: High-recall contextual regex parser for all statutory declarations with dynamic confidence scoring.
backend/extraction/patterns.py	Extraction	Compiled regex dictionaries PATTERNS and FALLBACK_PATTERNS for MRP, Net Qty, Dates, FSSAI, Consumer Care, Origin, Mfr.
backend/compliance/engine.py	Compliance	ComplianceEngine: Dispatches product declarations to codified rule evaluators, aggregates checks, and triggers recommendations.
backend/compliance/rules/legal_metrology.py	Compliance	Rule evaluators evaluate_lm_001..009 + compute_font_size_and_readability() for Rule 12 font heights.
backend/compliance/rules/fssai.py	Compliance	Rule evaluators evaluate_fs_001..005 for food commodities.
backend/compliance/scorer.py	Compliance	calculate_score(): Implements weighted compliance points formula and status classification.
backend/compliance/recommendations.py	Compliance	Deterministic corrective guidance engine mapping all 14 statutory rules to actionable legal advice.
backend/enforcement/penalties.py	Enforcement	estimate_penalty() (LM Act Sec 36/38 fines) and generate_show_cause() (statutory legal notice formatting).
backend/api/analyze.py	API	POST /api/analyze (multi-panel upload) & POST /api/analyze/text (raw text analysis).
backend/api/report.py	API	GET /api/report/{id} (PDF), /csv (CSV), /xlsx (openpyxl Excel), /json (JSON).
backend/services/report_service.py	Service	generate_pdf_report(): 768-line ReportLab PDF builder with styled tables, score badges, and legal citations.
frontend/src/App.tsx	Frontend	React Router v7 routes, RequireAuth and AdminOnly navigation guards, Theme/Auth/Role provider wrappers.
frontend/src/services/api.ts	Frontend	Centralized API client: login, register, getUsers, analyzeProducts, getHistory, estimatePenalty, extractText, checkCompliance.

<code>frontend/src/components/EvidenceViewer.tsx</code>	Frontend	Interactive packaging image viewer with scaled 2D bounding boxes, confidence badges, and panel switching.
<code>frontend/src/pages/Analyze.tsx</code>	Frontend	4-slot multi-panel packaging upload (Front, Back, Side 1, Side 2) with drag-and-drop and camera capture.
<code>frontend/src/pages/AnalyzeListing.tsx</code>	Frontend	E-commerce listing text screening mode with sample loader, declaration grid, and inline rule checklist.
<code>frontend/src/pages/Results.tsx</code>	Frontend	1771-line comprehensive audit dossier page: EvidenceViewer, score circle, rule matrix, penalty calculator, show-cause modal, multi-format export triggers.
<code>runner.py</code>	Automation	Unified multi-service runner: clears ports 5173/8000, detects LAN IP, prints QR code, and supervises backend+frontend processes.

6. REST API Specifications & Endpoints Reference

Comprehensive inventory of all backend REST endpoints hosted under the `/api` prefix.

HTTP	Endpoint Path	Auth Level	Request Payload / Parameters	Response Description
POST	<code>/api/analyze</code>	Public / Any	Multipart: files (1-4 images), labels (JSON array)	AnalysisResponse (complete audit payload)
POST	<code>/api/analyze/text</code>	Public / Any	JSON: { text: string }	AnalysisResponse (without images)
POST	<code>/api/ocr</code>	Public / Any	Multipart: file (single image)	OCRResult (raw text, words, bounding boxes)
POST	<code>/api/extract</code>	Public / Any	JSON: { text: string }	ProductInfo (extracted statutory fields)
POST	<code>/api/compliance/check</code>	Public / Any	JSON: ProductInfo object	ComplianceResult (checks, score, recs)
GET	<code>/api/compliance/rules</code>	Public / Any	None	List of all 14 RuleDefinition objects
GET	<code>/api/demo/cases</code>	Public / Any	None	Catalog of 3 benchmark test cases
GET	<code>/api/demo/{case_id}</code>	Public / Any	Path: case_id ('1', '2', or '3')	In-memory demo AnalysisResponse
GET	<code>/api/history</code>	Bearer Token	None	List of real user screening HistoryItem records
GET	<code>/api/history/{id}</code>	Public / Any	Path: id (UUID or demo ID)	AnalysisResponse for requested ID
DELETE	<code>/api/history/{id}</code>	Officer / Admin	Path: id	Deletes analysis record and image files
DELETE	<code>/api/history</code>	Admin Only	None	Purges all user analysis history from database
GET	<code>/api/stats</code>	Bearer Token	None	DashboardStats (counts, avg score, recent)
GET	<code>/api/stats/trends</code>	Bearer Token	Query: days (int, 7-90, default 14)	Daily compliance counts for Recharts trends
GET	<code>/api/report/{id}</code>	Public / Any	Path: id	Downloadable ReportLab PDF document
GET	<code>/api/report/{id}/xlsx</code>	Public / Any	Path: id	Downloadable openpyxl 3-sheet Excel (.xlsx)
GET	<code>/api/report/{id}/csv</code>	Public / Any	Path: id	Downloadable CSV spreadsheet report
POST	<code>/api/enforcement/penalty</code>	Officer / Admin	JSON: { analysis_id, repeat_offence, prior_notices }	PenaltyEstimate (Sec 36/38 fine bracket)
POST	<code>/api/enforcement/notice</code>	Officer / Admin	JSON: { analysis_id, notice_reference, officer_name }	ShowCauseNotice (formatted legal notice)
POST	<code>/api/auth/login</code>	Public / Any	JSON: { username, password }	AuthResponse: { token, user: UserOut }
POST	<code>/api/auth/register</code>	Public / Any	JSON: { username, password, full_name, role }	AuthResponse (creates public merchant)
POST	<code>/api/auth/users</code>	Admin Only	JSON: { username, password, role, jurisdiction }	Creates officer or merchant account
PUT	<code>/api/auth/users/{username}</code>	Admin Only	JSON: { full_name, jurisdiction, role, new_password }	Updates user account / resets password
DELETE	<code>/api/auth/users/{username}</code>	Admin Only	Path: username	204 No Content (deletes user account)
GET	<code>/api/health</code>	Public / Any	None	{ status: 'healthy', ocr_available, engine }

CONCLUSION & REGULATORY IMPACT:

MetrCheck AI delivers an objective, evidence-backed automated compliance pipeline that scales legal metrology and food safety inspections from minutes per package to sub-second processing. By marrying computer vision OCR with deterministic legal rule evaluation and formal enforcement tools, it equips enforcement officers with court-ready evidence while providing manufacturers with clear, actionable guidance to correct labeling non-compliances prior to market entry.